

Desenvolvimento de Aplicações com Node.js



Conteudista: Prof. Esp. Jônatas Carvalho

Revisão Textual: Esp. Anna Carolina Guimarães

Objetivo da Unidade:

- Capacitar o aluno no uso de *pipelines* CI/CD com a ferramenta Jenkins e organização de projetos em Node.js de forma escalável utilizando as boas práticas de desenvolvimento.

  Material Teórico

  Referências



Material Teórico

O que é CI/CD (Integração Contínua/ Implantação Contínua)?

Entrega contínua ou integração contínua é o processo de integrar modificações da *codebase* de forma contínua e automatizada, podendo, assim, evitar erros humanos de verificação, garantindo mais segurança e agilidade no desenvolvimento de um software.

Imagine que possuímos uma equipe com 3 desenvolvedores (Dev A, Dev B, Dev C) e foi estabelecida uma nova tarefa para o time, a tarefa seria desenvolver uma nova *feature* para a aplicação em que a equipe trabalha e, então, enviar essa *feature* para o ambiente produtivo sem quebrar os testes e as *features* que já existem na aplicação. Sem a entrega contínua como seria feita essa tarefa?

- Dev A desenvolveria a aplicação;

- Dev B e Dev C fariam o *code review*, as verificações de qualidade de código, verificações de segurança e outras verificações;
- Dev A, então, faria o *deploy* no ambiente produtivo.

Com a integração contínua, como seria feita essa tarefa?

- Dev A desenvolveria a aplicação;
- Dev B e Dev C faria o *code review*;
- CI faria todas as verificações e, então, faria o *deploy*.

Perceba que, com a integração contínua, ela executaria diversos passos como: execução de testes, verificações de qualidade de código e de segurança, geração de artefatos prontos para o processo de *deploy*, identificação de próxima versão, geração de *tags* e *releases*, tudo isso de uma forma automática e sem possibilidade de erros humanos, o que pode acontecer frequentemente quando não utilizamos ferramentas de CI.

Já CD – *Continuous Deployment* ou Implantação Contínua – é basicamente uma extensão da Integração Contínua na qual a parte de implantação, ou seja, o *deploy* das aplicações é realizada de forma automatizada, fazendo com que as alterações de código que necessitam de aprovação, sejam automaticamente aprovadas e implantadas em produção sem a necessidade de interação humana.

Esse tipo de automatização deve ser muito bem construído para evitar que *bugs* passem a integrar um ambiente produtivo, mas se bem construído traz diversos benefícios como a redução de tempo de entrega e lançamentos com maior frequência e menos esforço.

Ferramentas de CI

Existem diversas ferramentas de CI/CD e, aqui, abordaremos as 3 principais que são Travis, Jenkins e GitLab CI/CD.

- **Travis CI:** é uma das ferramentas de integração contínua que é hospedada na nuvem. É muito popular, pois é utilizada em projetos de código aberto. Possui um grande suporte em diversas linguagens de programação e, também, com repositórios como GitHub e Bitbucket. Possui uma configuração simplificada porque utiliza arquivos yaml para definir os fluxos de trabalho das **pipelines**;
- **Jenkins:** é uma ferramenta *open source*, que pode criar ambientes hospedados na nuvem ou localmente. O Jenkins possui uma configuração inicial mais dificultosa, mas é muito utilizado no mercado porque possui uma grande versatilidade e uma grande capacidade de personalização;
- **GitLab CI/CD:** é uma ferramenta que é integrada ao GitLab, sua principal qualidade também é sua principal falha, que é a

integração apenas com o GitLab. Possui uma grande simplicidade para configuração.

Glossário

Pipeline: é uma automação que realiza o gerenciamento de um projeto, desde a construção do código até a implantação do projeto em um ambiente produtivo.

A Tabela abaixo mostra as principais características de cada uma dessas ferramentas:

Tabela 1 – Comparação entre Travis, Jenkins e GitLab

Critério	Travis	Jenkins	GitLab CI/CD
Tipo de Hospedagem	É hospedado na nuvem	Pode ser hospedado na nuvem ou local	É hospedado no próprio ambiente do GitLab

Configurações iniciais	É bem simples e é construída dentro de um arquivo <code>.travis.yml</code>	É um pouco complexa pois exige instalação e configuração, que em empresas é normalmente construída por equipes de DevOps	É simples, construída dentro de um arquivo <code>.gitlab-ci.yml</code>
Gastos	Para projetos <i>open-source</i> não possui custos, porém é paga para projetos privados	Totalmente gratuito, o único custo fica em relação a hospedagem	Está incluído nos planos que são oferecidos pelo GitLab
Linguagens	Aceita a maioria das linguagens relevantes de mercado	Aceita a maioria das linguagens pois a comunidade e os próprios desenvolvedores do Jenkins cria <i>plugins</i> para	Pela integração que possui com o Docker aceita também uma grande maioria das linguagens de mercado

		essas integrações	
<i>Plugins</i>	Comparado ao Jenkins possui uma biblioteca de <i>plugins</i> bem limitada	Possui uma biblioteca extensa com mais de 1.800 <i>plugins</i> disponíveis	É limitado pois os <i>plugins</i> estão ligados à funcionalidades do GitLab
Interface	Possui uma interface bem intuitiva e simples de utilizar	É um pouco confusa, principalmente para iniciantes no uso da ferramenta	Possui uma interface bem intuitiva e simples de utilizar
Velocidade	Possui uma rápida execução	Depende dos recursos locais	É escalável com <i>runners</i> que são uma ferramenta GitLab, podendo aumentar a quantidade conforme a necessidade

Containers	Possui suporte a <i>containers</i>	Precisa de <i>plugins</i> para o suporte a <i>containers</i>	É nativo, pois o GitLab possui suporte direto ao Docker
Comunidade	Muito popular entre projetos open-source	Possui uma comunidade imensa e madura	É uma comunidade crescente especialmente para DevOps
Repositórios	Utiliza principalmente GitHub e Bitbucket	Pode ser utilizado no GitHub, Bitbucket e também GitLab através do uso dos <i>plugins</i>	Integrado nativamente ao GitLab

Reflita

Suponha que iremos criar 3 projetos que possuem diferentes portes (pequeno, médio e grande). Para cada projeto, qual ferramenta você utilizaria e por quê?

Criando um Container Jenkins

Criar uma *pipeline* traz uma infinidade de possibilidades, pois o próprio Jenkins tem integração com diversas linguagens de programação, sistemas de controle de versão, gerenciadores de testes entre outras coisas. Mas que tal dentro das nossas limitações criarmos nossa própria *pipeline*?

Lembram do nosso projeto do conversor de números binários para decimais? Vamos utilizá-lo para criar nossa *pipeline* utilizando o Jenkins. Anteriormente, criamos o projeto e subimos ele para o GitHub, com isso vamos utilizar o link do repositório que deve ser algo parecido como:

https://github.com/nome_do_seu_usuario/nome_do_projeto.

Não iremos realizar a instalação das ferramentas em nossa máquina, vamos utilizar o Docker para realizar a execução do Jenkins. Por isso, crie um diretório em seu computador e, então, crie um arquivo chamado: `docker-compose.yml`. No arquivo, insira o código abaixo:

```
version: '3.8'
services:
  jenkins:
    build: . # Refere-se ao diretório contendo o Dockerfile
    container_name: jenkins
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    environment:
      - JAVA_OPTS=-Djenkins.install.runSetupWizard=false
    volumes:
      jenkins_home:
```

Basicamente estamos criando um *container* que roda o Jenkins, gostaria que você leitor prestasse atenção nas linhas:

- **build:** **∴** esse ponto informa que o Dockerfile está no mesmo diretório do nosso *compose*;
- **ports: 8080:8080:** essa linha está mapeando a porta 8080 que está interna no *container* Jenkins para uma porta externa 8080, em resumo, estamos deixando a porta 8080 liberada para que alguém fora do *container* possa acessá-la, ou seja, para acessarmos o Jenkins, basta acessarmos o endereço: localhost:8080 em nosso navegador.

Agora vamos criar nosso arquivo Dockerfile, o nome deve ser exatamente esse e ele não possui extensão. Criando esse arquivo, basta inserir o código abaixo:

```
FROM jenkins/jenkins:lts
USER root
# Instalar NodeJS
RUN curl -fsSL https://deb.nodesource.com/setup_23.x | bash - \
    && apt-get install -y nodejs \
    && apt-get clean
# Retorne o controle para o usuário Jenkins
USER jenkins
```

Os códigos acima apontam para a imagem do Jenkins, então muda o usuário para o *root*, que é o equivalente ao usuário administrador do Windows, então ele roda três comandos que basicamente vão fazer o download do Node.js e faz a instalação e, por último, ele retorna para o usuário padrão do Jenkins.

Feito isso, temos agora nossa estrutura para subir o *container*. Basta abrir o terminal no diretório que você criou anteriormente e digitar o comando:

- **docker-compose up -d.**

Após a execução do comando você deve receber um retorno próximo a imagem abaixo. Isso quer dizer que o *container* está

funcionando normalmente.

```
→ docker-compose up -d  
Creating network "dockerizadojenkins_default" with the default driver  
Creating jenkins ... done
```

Figura 1 – Saída no terminal indicando que o *container* subiu corretamente

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem ilustra a saída que o Docker emite no terminal. Assim que o *container* sobe, ela exibe a mensagem de que foi criada a network e, também, sinaliza que criou o Jenkins. Fim da descrição.

Utilizando *Plugins* na Construção de uma *Pipeline*

Agora, abra seu navegador e digite em sua barra de navegação o endereço: **localhost:8080**. Feito isso, você vai estar na página inicial do Jenkins, como na imagem abaixo:

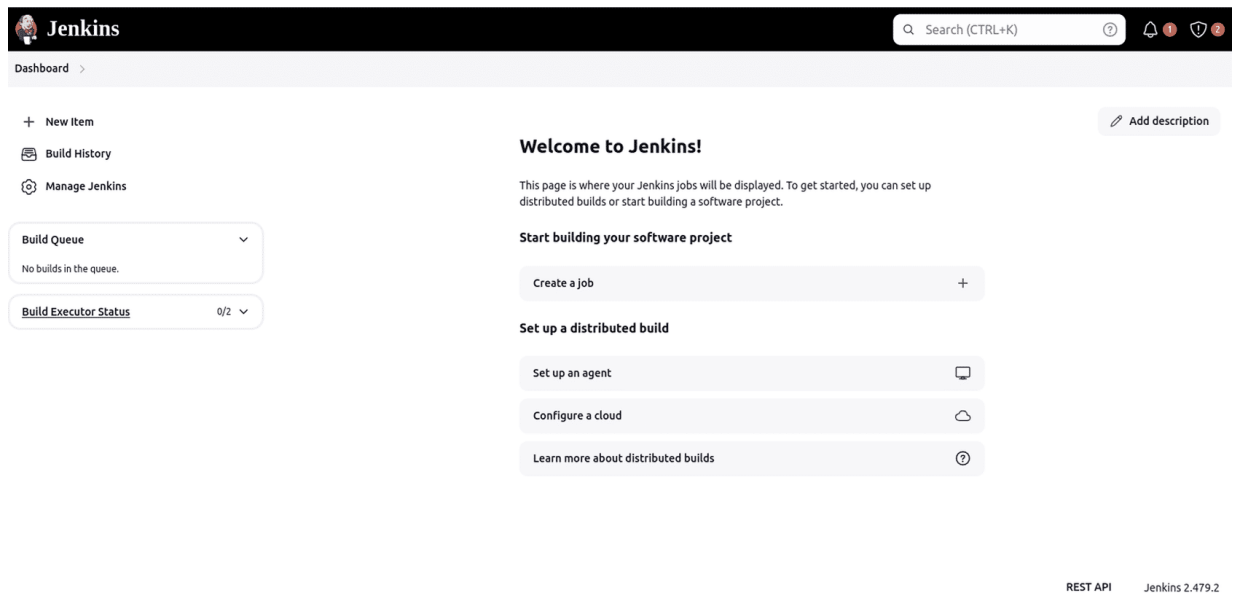


Figura 2 – Jenkins

Fonte: Acervo do Conteudista

#ParaTodosVerem: a imagem ilustra a página inicial da ferramenta Jenkins. Na parte superior, há uma barra de busca. Na esquerda, alguns menus como *New Item*, *Build History*, *Manage Jenkins*. Na parte central, há uma mensagem de boas-vindas e alguns menus de configurações de *build*. Fim da descrição.

Clique no botão *Manage Jenkins* (Gerenciar Jenkins) > *Plugins* (Gerenciar *Plugins*) > *Available plugins* (*Plugins* disponíveis). Vamos instalar os seguintes *plugins*: *Pipeline*, *Pipeline Stage View*, *Node.js*, *JUnit*, *HTML Publisher*, *Git*, *GitHub*.

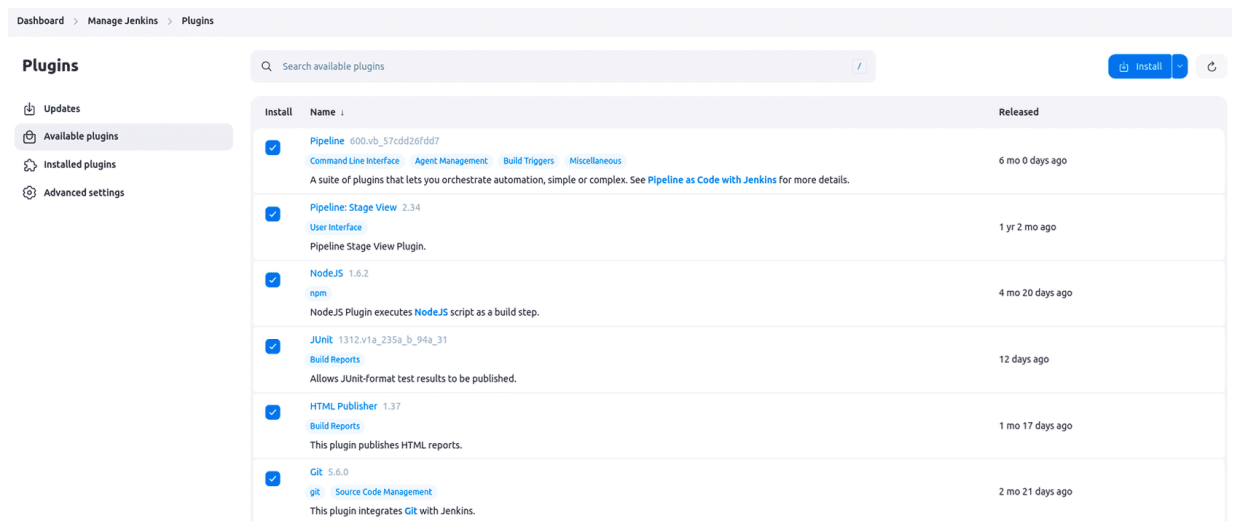


Figura 3 – Plugins Jenkins

Fonte: Acervo do Conteudista

#ParaTodosVerem: a imagem ilustra a página de download de *plugins* do Jenkins. Ao lado esquerdo, exibe um menu com os itens *Updates*, *Available plugins*, *Installed plugins*, *Advanced settings*. Na parte central, apresenta os *plugins* que estamos realizando a instalação, como: *Pipeline*, *Pipeline Stage View*, *Node.js*, *JUnit*, *HTML Publisher* e *Git*. Fim da descrição.

Uma breve explicação para que cada *plugin* seja utilizado:

- **Pipeline:** este *plugin* permite a possibilidade de criar e realizar o gerenciamento das *pipelines* via código, podendo ser via script dentro do console do próprio Jenkins ou via script local criando um Jenkinsfile;
- **Pipeline Stage View:** ao iniciar nossas *pipelines*, este *plugin* vai gerar uma interface gráfica para a execução de nossas *pipelines*, mostrando cada estágio como uma etapa visualmente;

- **Node.js:** como trabalhamos com Node.js em nosso projeto, precisamos gerenciar esta linguagem dentro do Jenkins; por isso, vamos utilizar este *plugin*;
- **JUnit:** este *plugin* realiza a análise de relatórios de ferramentas de teste como o Jest que usamos em nosso projeto;
- **HTML Publisher:** os resultados dos testes são exibidos via *logs*, então para deixar esses *logs* mais humanizados e mais simples de ler vamos utilizar esse *plugin*;
- **Git:** faz a integração do Jenkins com o Git;
- **GitHub:** é o *plugin* que integra o Jenkins com os repositórios.

Em resumo, os *plugins* permitem que criemos *pipelines* modernas, integradas com repositórios e ferramentas, executando testes e deixando os resultados mais humanizados.

Criação de *Pipeline* Utilizando Jenkins

Vamos agora criar nossa *pipeline*, volte para a página inicial do Jenkins. Clique no botão no menu à esquerda + *New Item*. Agora escreva um nome para sua *pipeline* e selecione a opção *Pipeline* e clique no botão OK.

New Item

Enter an item name

pipeline_de_explicacao_jenkins

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Figura 4 – Criação de *pipeline* Jenkins

Fonte: Acervo do Conteudista

#ParaTodosVerem: a imagem ilustra a página em que criamos a *pipeline* no Jenkins. Ela possui quatro opções de escolha: *Freestyle project*, *Pipeline*, *MultiConfiguration project* e *Folder*. Na imagem, a opção *Pipeline* está selecionada e, ao fim da página, temos um botão em azul Ok. Fim da descrição.

Após criar a *pipeline*, clique em *Configure* (Configurar) no menu à esquerda, ao clicar o navegador vai abrir uma nova página. Desça até a seção *Pipeline*, selecione a opção *Pipeline script* e dentro da caixa denominada como Script insira o código abaixo, depois clique no botão *Save* (Salvar).


```

pipeline
  agent any
  stages
    stage (' Checkout (Git)')
      steps
        git url:
          'https://github.com/nome_do_seu_usuario/nome_do_seu_projeto', branch:
          'main'
    }
    stage('Instalando Dependências')
      steps
        sh 'npm install'
    }
    stage('Rodar Testes')
      steps
        sh 'npm test'
    }
    stage('Cobertura de Testes')
      steps {
        sh 'npm run coverage'
      }
    }
    stage('Relatório JUnit')
      steps
        sh 'npm run test:ci'
        junit 'reports/junit.xml'
    }
    stage('Publicar Relatório de Cobertura')
      steps
        publishHTML(target: [
          reportName: 'Coverage Report',
          reportDir: 'coverage/lcov-report',
          reportFiles: 'index.html'
        ])
    }
  }
  post
    always
      cleanWs()

```

```
}  
}  
}
```

Após isso, basta clicar no botão *Build Now* no menu à esquerda e, então, a *pipeline* vai fazer seu trabalho executando todos seus estágios.

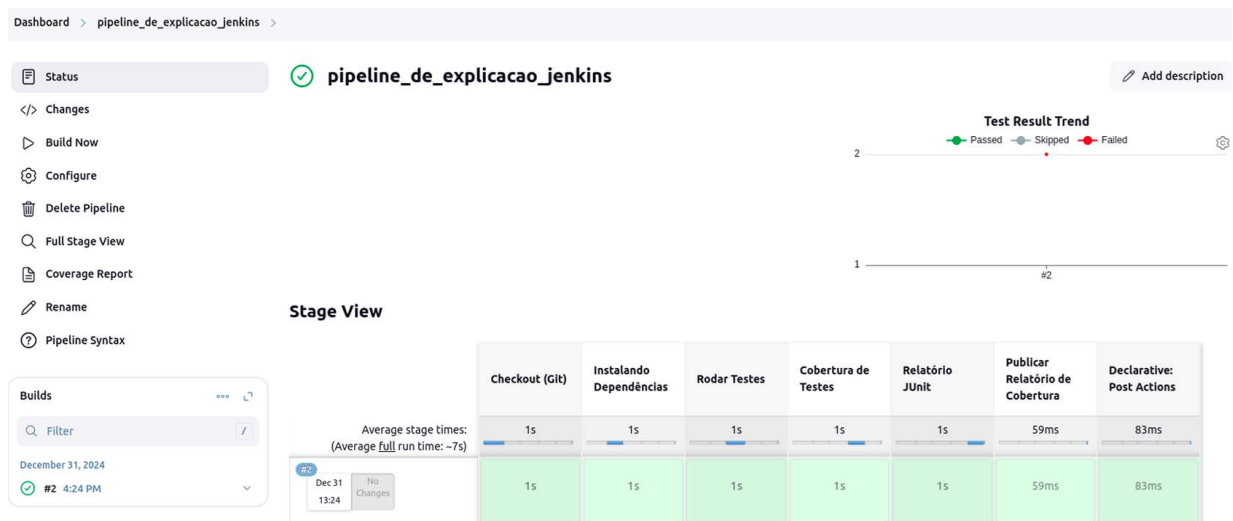


Figura 5 – Exibição dos estágios da *pipeline* no Jenkins

Fonte: Acervo do Conteudista

#ParaTodosVerem: a imagem exibe os estágios da *pipeline* que acabamos de criar. A *pipeline* possui 7 estágios: Checkout, Instalando Dependências, Rodar Testes, Cobertura de Testes, Relatório JUnit, Publicar Relatório de Cobertura. Fim da descrição.

Agora vamos a explicação de cada estágio:

- **Checkout (GIT):** realiza *checkout* do repositório informando, faz o download da *branch* para o *workspace* do Jenkins;
- **Instalando dependências:** basicamente executa o comando *npm install* e instala as dependências que nosso projeto necessita para rodar;
- **Cobertura de testes:** executa o comando *npm run coverage* que irá gerar os relatórios detalhando quais partes do código estão cobertos pelos testes;
- **Relatório JUnit:** exibe os relatórios de testes a partir do arquivo *reports/junit.xml*;
- **Publicar relatório de cobertura:** cria um arquivo HTML tornando os *logs* dos testes em uma interface para exibição.

Apesar de criarmos uma *pipeline* com poucas etapas, criamos uma *pipeline* que é bem funcional e pode ser utilizada em projetos na vida real, pois ela se integra com Git e GitHub, clona o projeto, realiza e verifica os testes e exibe a cobertura.

Visualização da Cobertura de Testes

Para acessar o relatório de cobertura gerado pela nossa *pipeline* criada no Jenkins, basta digitar o seguinte endereço:

http://localhost:8080/job/pipeline_de_explicacao_jenkins/Cove

rage_20Report/. Neste endereço, você terá uma visualização parecida com a da imagem abaixo:

All files

100% Statements 14/14 100% Branches 2/2 100% Functions 2/2 100% Lines 14/14

Press n or j to go to the next uncovered block, b, p or k for the previous block.

File		Statements	Branches	Functions	Lines
app.js		100%	7/7	100% 0/0	100% 1/1 7/7
validateDecimal.js		100%	7/7	100% 2/2	100% 1/1 7/7

Figura 6 – Exibição do relatório de testes

Fonte: Acervo do Conteudista

#ParaTodosVerem: a imagem exibe o relatório dos testes que criamos, exibindo a cobertura das *branches*, funções e linhas do nosso código. Fim da descrição.

Perceba que o relatório exibe nossas duas classes criadas em nosso projeto, a cobertura das *branches*, das funções e das linhas, e, em nosso caso, é 100% em todas as categorias. Uma dica de estudo seria alterar nossos testes para mudar a cobertura das *branches*, funções e linhas.



Referências

GITLAB. **Documentação oficial:** CI/CD. Disponível em:
<<https://docs.gitlab.com/ee/ci>>. Acesso em: 28/12/2024.

JENKINS. **Documentação oficial.** Disponível em:
<<https://www.jenkins.io/doc>>. Acesso em: 28/12/2024.

KANE, S. P.; MATTHIAS, K. *Docker: up & running*. 3. ed.
Sebastopol: O'Reilly Media, 2023.

SMART, J. F. *Jenkins: the definitive guide*. Sebastopol: O'Reilly
Media, 2011.

TRAVIS CI. **Documentação oficial.** Disponível em:
<<https://docs.travis-ci.com>>. Acesso em: 28/12/2024.